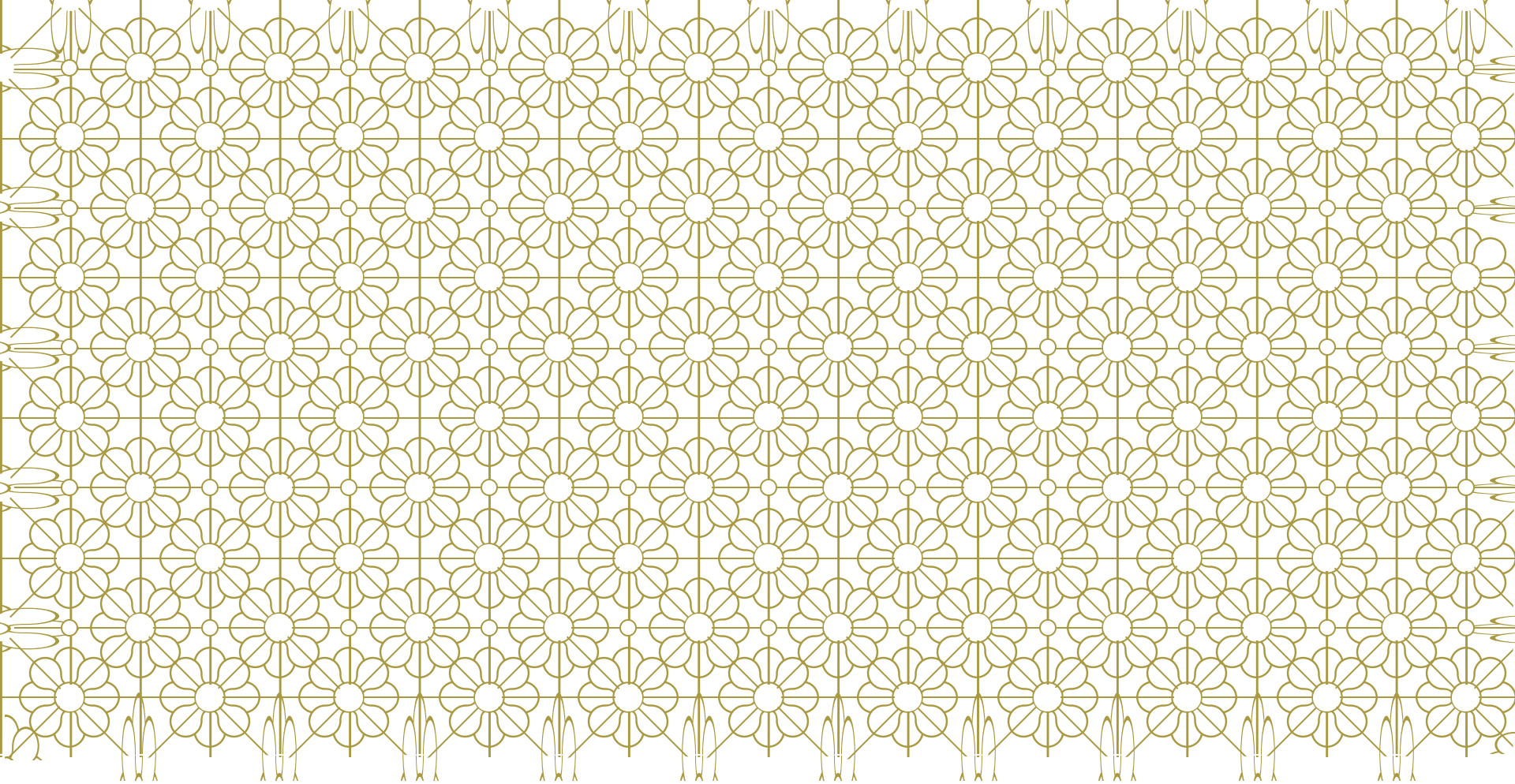
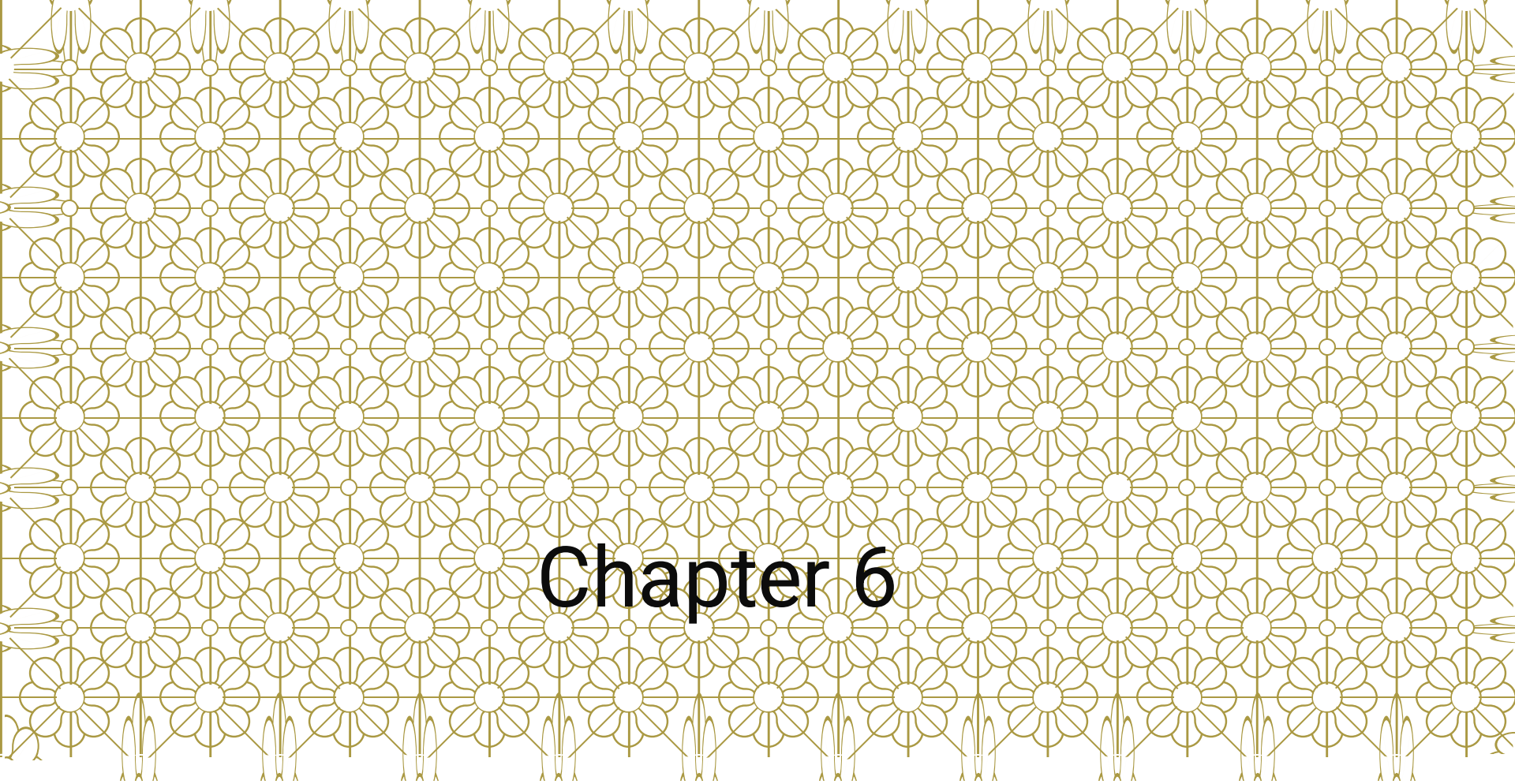




# Algorithms analysis and Design

By : Dr. Basem  
Alijla



## Chapter 6

# Transform and Conquer

# OUTLINE

## Introduction to Transform and Conquer technique

### instance simplification

- Presorting
- Gaussian Elimination

### representation change

- Balanced Search Trees
- Heaps and Heapsort

# Transform and Conquer

- This group of techniques solves a problem by transformation
- Two Stage procedure
  1. Transform
    - The problem's instance is modified to be more amenable to solution.
  2. Conquer

Solve the Problem

# Transform and Conquer

- There are three major variations of transformation

# Transform and Conquer

- There are three major variations of transformation
- instance simplification : transform to a simpler/more convenient instance of the same problem
- representation change: transform to a different representation of the same instance
- problem reduction: transform to a different problem for which an algorithm is already available

# OUTLINE

Introduction to Transform and Conquer technique

instance simplification

- Presorting
- Gaussian Elimination

representation change

- Balanced Search Trees
- Heaps and Heapsort

# Instance simplification - Presorting

- Solve a problem's instance by transforming it into another simpler/easier instance of the same problem
- Presorting:
- Many problems involving lists are easier when list is sorted
  1. searching
  2. checking if all elements are distinct (element uniqueness)
  3. Computing the mode of a list



# How fast can we sort ?

- Efficiency of algorithms involving sorting depends on efficiency of sorting.
  - Selection and Insertion Sort  $O(n^2)$
  - Bubble Sort  $O(n^2)$
  - Quick and Merge Sort  $O(n \log n)$

# How fast can we sort ?

- Theorem :  $n \log_2 n$  comparisons are necessary in the worst case to sort a list of size  $n$  by any comparison-based algorithm
- Note: About  $n \log_2 n$  comparisons are also sufficient to sort array of size  $n$  (**mergesort**).

# Searching with presorting

- Problem: Search for a given  $K$  in  $A[0..n-1]$
- Presorting-based algorithm:
  - Stage 1 : Sort the array by an efficient sorting algorithm
  - Stage 2 : Apply binary search

Efficiency:  $\Theta(n \log n) + O(\log n) = \Theta(n \log n)$

Good or bad?

Why do we have our dictionaries, telephone directories, etc. sorted?

# Example: Searching

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 6 | 4 | 8 | 2 | 9 | 3 | 5 | 1 | 7 |
|---|---|---|---|---|---|---|---|---|

- Sort the array by Merge sort algorithm (  $O(n \log n)$  )

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|



- the Key Value ( 7 ) using Binary Search (  $\log n$  )

|   |   |   |   |
|---|---|---|---|
| 6 | 7 | 8 | 9 |
|---|---|---|---|



- Found !

# Element Uniqueness with presorting

- Presorting-based algorithm
  - Stage 1 : Sort the array by an efficient sorting algorithm
  - Stage 2: scan array to check pairs of adjacent elements

Efficiency:  $\Theta(n \log n) + O(n) = \Theta(n \log n)$

- Brute force algorithm : Compare all pairs of elements
- Efficiency:  $O(n^2)$

# Example:

- Sort the array by Merge sort algorithm (  $O(n \log n)$  )

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 6 | 4 | 8 | 2 | 9 | 3 | 5 | 1 | 3 |
|---|---|---|---|---|---|---|---|---|

- Compare the Adjacent pairs  $O(n)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 3 | 4 | 5 | 6 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|



|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 3 | 4 | 5 | 6 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|



|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 3 | 4 | 5 | 6 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|



# Computing a mode with presorting

A **mode** is a value that occurs most often in a given list of numbers

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 6 | 4 | 6 | 2 | 9 | 6 | 5 | 2 | 5 |
|---|---|---|---|---|---|---|---|---|

❖ The Mode is 6 .

**The brute-force approach:  $O(n^2)$**

- scan the list and compute the frequencies of all its distinct values
- $c(n) = \sum_{i=1}^n (i - 1) = 0 + 1 + 2 + \dots + (n - 1) = \frac{(n-1)n}{2}$
- find the value with the largest frequency
  - $(n - 1)$

# Computing a mode with presorting

- Presorting-based algorithm
  - Stage 1 : Sort the array by an efficient sorting algorithm
  - Stage 2: Find the longest run of adjacent equal values in the array

Efficiency:  $\Theta(n \log n) + O(n) = \Theta(n \log n)$



# Example: Computing a mode

- Sort the array by Merge sort algorithm (  $O(n \log n)$  )

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 6 | 4 | 6 | 2 | 9 | 6 | 5 | 2 | 5 |
|---|---|---|---|---|---|---|---|---|

- Compare the Adjacent pairs  $O(n)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 2 | 2 | 4 | 5 | 5 | 6 | 6 | 6 | 9 |
|---|---|---|---|---|---|---|---|---|



|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 2 | 2 | 4 | 5 | 5 | 6 | 6 | 6 | 9 |
|---|---|---|---|---|---|---|---|---|



|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 2 | 2 | 4 | 5 | 5 | 6 | 6 | 6 | 9 |
|---|---|---|---|---|---|---|---|---|



|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 2 | 2 | 4 | 5 | 5 | 6 | 6 | 6 | 9 |
|---|---|---|---|---|---|---|---|---|



# Computing a mode with presorting

# OUTLINE

Introduction to Transform and Conquer technique

instance simplification

- Presorting
- Gaussian Elimination

representation change

- Balanced Search Trees
- Heaps and Heapsort

# Instance simplification – Gaussian Elimination

- Given: A system of **n linear equations** in **n unknowns** with an arbitrary coefficient matrix.
- Transform to: An equivalent system of **n linear equations** in **n unknowns** with an **upper triangular coefficient matrix**.
- Solve the latter by substitutions starting with the last equation and moving up to the first one.

# Gaussian Elimination

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

.....

$$a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n$$



$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \quad b_1$$

$$+ a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$+ a_{nn}x_n = b_n$$

# Gaussian Elimination

- The transformation is accomplished by a sequence of elementary operations on the system's coefficient matrix (which don't change the system's solution):
- for  $i = 1$  to  $n-1$  do
  - replace each of the subsequent rows (i.e., rows  $i+1, \dots, n$ ) by
    - a difference between that row and an appropriate multiple
    - of the  $i$ -th row to make the new coefficient in the  $i$ -th column

# Example of Gaussian Elimination

- Solve 
$$\begin{array}{rcl} 2x_1 - 4x_2 + x_3 & = & 6 \\ 3x_1 - x_2 + x_3 & = & 11 \\ x_1 + x_2 - x_3 & = & -3 \end{array}$$

- coefficient matrix

$$\begin{array}{cccc} 2 & -4 & 1 & 6 \\ 3 & -1 & 1 & 11 \\ 1 & 1 & -1 & -3 \end{array}$$

# Example of Gaussian Elimination

- Gaussian elimination

$$\begin{array}{cccc}
 2 & -4 & 1 & 6 \\
 3 & -1 & 1 & 11 & \text{row2} - (3/2)*\text{row1} \\
 1 & 1 & -1 & -3
 \end{array}$$

$$\begin{array}{cccc}
 2 & -4 & 1 & 6 \\
 0 & 5 & -\frac{1}{2} & 2 \\
 1 & 1 & -1 & -3 & \text{row3} - (1/2)*\text{row1}
 \end{array}$$

$$\begin{array}{cccc}
 2 & -4 & 1 & 6 \\
 0 & 5 & -\frac{1}{2} & 2 \\
 1 & 1 & -1 & -3 \\
 2 & -4 & 1 & 6 \\
 0 & 5 & -\frac{1}{2} & 2 \\
 0 & 3 & -\frac{3}{2} & -6
 \end{array}$$



# Example of Gaussian Elimination

$$\begin{array}{cccc} 2 & -4 & 1 & 6 \\ 0 & \color{red}{5} & -\frac{1}{2} & 2 \\ 0 & \color{red}{3} & -\frac{3}{2} & -6 \end{array} \quad \text{row3} - (3/5)*\text{row2}$$

$$\begin{array}{cccc} 2 & -4 & 1 & 6 \\ 0 & \color{red}{5} & -\frac{1}{2} & 2 \\ 0 & 0 & -\frac{6}{5} & -\frac{36}{5} \end{array}$$

upper triangular coefficient matrix.

## Backward substitution

$$-\frac{6}{5}x_3 = -\frac{36}{5} \quad 6x_3 = 36 \quad , \quad x_3 = 6$$

$$5x_2 - \frac{1}{2}x_3 = 2 \quad 5x_2 = 2 + 3 \quad , \quad x_2 = 1$$

$$2x_1 - 4x_2 + x_3 = 6 \quad 2x_1 = 6 - 2 \quad , \quad x_1 = 2$$

# Pseudocode and Efficiency of Gaussian Elimination

Stage 1: Reduction to an upper-triangular matrix

for i    1 to n-1 do

    for j    i+1 to n do

        for k    i to n+1 do

$A[j, k] \leftarrow A[j, k] - A[i, k] * A[j, i] / A[i, i]$

$$\Theta(n^3)$$

# Pseudocode and Efficiency of Gaussian Elimination

Stage 2: Back substitutions

for j ← n downto 1 do

    t ← 0

$\Theta(n^2)$

    for k ← j + 1 to n do

        t ← t + A[j, k] \* x[k]  
    x[j] ← (A[j, n+1] - t) / A[j, j]

Efficiency:  $\Theta(n^3) + \Theta(n^2) = \Theta(n^3)$

# Assignment 1

- Write An Algorithm that compute the mode in a list with complexity less than  $n \log n$  , in other word with a complexity not grater than  $O(n)$

# OUTLINE

Introduction to Transform and Conquer technique

instance simplification

- Presorting
- Gaussian Elimination

representation change

- Balanced Search Trees
- Heaps and Heapsort